

Technical Interview: Embedded Firmware Engineer

Firmware, Real-Time Control & Systems Integration

Free-Space Optical Communications

All three parts are mandatory: Question 1, Question 2, and the Grand Challenge.

Ground Rules

- Every part number you select must be accompanied by the datasheet section, table, or parameter that decides the choice. "A suitable ADC" scores zero.
- Every numeric answer must be marked calculated, simulated, measured, or estimated.
- Where a specification in this paper is inconsistent, marginal, or unachievable, say so and reconcile it. Some of them are. Flagging one with the arithmetic that proves it scores higher than silently designing around it.
- Where this paper names an operating system, processor, or protocol, treat it as the previous engineer's choice and not as a constraint on yours. If you change it, say why.

The System You Are Building

QOSMIC's optical ground station terminal establishes a 2.5 - 10 Gbps bidirectional laser link with LEO satellites at 500–2000 km range, over passes lasting 5–10 minutes. An Alt-Azimuth mount performs coarse pointing from ephemeris. A piezo fast steering mirror (FSM) performs fine pointing and rejects atmospheric tip-tilt. A quadrant photodetector (QPD) supplies the beam position error, and a fibre coupling stage delivers the received beam into single-mode fibre. Transmit is a 1550nm DFB seed amplified by an EDFA. The terminal runs unattended outdoors at 10–55 °C, IP65, from a 48 V DC bus, with 12 months between service visits.

The same electronics core is intended to serve a space-side terminal on a CubeSat bus, where the host interface, power budget, and recovery options are all different. Question 1 is the FSM node. Question 2 is the gimbal node and the Linux supervisor. The Grand Challenge integrates both with the remaining subsystems, and asks what integration would have taught you.

Question 1: Fast Steering Mirror Node — Real-Time Firmware & Board Review

During a pass the atmosphere moves the received beam by tens of microradians at bandwidths extending to 300 Hz — far faster than a 200 mm telescope mount can follow. The FSM rejects this residual motion and holds the beam on the fibre coupling stage, closing on the QPD error signal with a strain-gauge inner loop. If this node misses its deadline the link drops.

Hardware as specified by the engineer who left. Piezo tip/tilt platform, two axes, $C = 3.5 \mu\text{F}$ per axis small-signal, 0–150 V unipolar, 4 mrad mechanical range, first structural resonance $f_n = 1.2 \text{ kHz}$ at damping ratio $\zeta = 0.02$. External HV amplifier, gain 20 V/V, 500 mA peak output, 30 kHz small-signal bandwidth. Position feedback from strain gauges read through an Analog Devices AD7386 simultaneous-sampling 16-bit SAR ADC over SPI. Command output through an AD5686R 16-bit quad DAC over SPI. Controller is an STM32H743ZI at 480 MHz. Their firmware ran FreeRTOS. Control loop 20 kHz hard real-time, targeting 500 Hz closed-loop bandwidth with phase margin $\geq 45^\circ$ and gain margin $\geq 10 \text{ dB}$. A calibration mode dithers the mirror over full stroke, 0–100 V, at up to 800 Hz. Board supply 24 V DC.

(a) Architecture, Scheduling & Supervision:

- Accept or reject the STM32H743ZI for this node, defending against the TI TMS320F28379D and the NXP i.MX RT1170 with the specific parameter that decides each comparison. Then make the scheduling decision independently: bare-metal super-loop, FreeRTOS, Zephyr, or another RTOS. State what an

RTOS earns you on a node whose dominant activity is a single 20 kHz loop, what it costs you in worst-case latency and jitter, and whether this node justifies one at all.

- Give the clock tree — HSE source, PLL, SYSCLK, bus prescalers — and the resulting SPI kernel clock and achievable SCK for the ADC and DAC. Then give the memory map: physical region and address for the DMA ADC buffer, the DMA DAC buffer, the control state block, and the hot ISR code. State which DMA controller instances on this device can and cannot reach each region, name the exact RCC register bits required before your first DMA transfer, and state the failure symptom if you omit them.
- State your D-cache configuration. If enabled, give the cache maintenance calls, the alignment and size constraints on every DMA buffer, and where in the data flow each call sits. Then give the NVIC priority table for the control timer, ADC DMA complete, SPI error, comms, and fault line, including the priority grouping you configure. If you chose an RTOS, state which of these ISRs may call its API and the configuration value that draws that line.
- Specify the watchdog: part number, window timing arithmetic, the exact firmware location of the refresh, and the condition that gates it. Justify why that condition proves the system is alive rather than merely still servicing interrupts.

(b) Drivers & Real-Time Implementation:

- The AD7386 is specified above for two strain channels, with the stated intent to expand to four channels on the next board revision. Verify this against the datasheet. State whether the part supports it, and if not, name the correct part and the throughput consequence.
- Write the SPI and DMA driver for the ADC at register level: conversion trigger, chip select timing, frame format, DMA configuration. Quote t_{CONVERT} , t_{ACQUIRE} and t_{CYC} from the datasheet and show that your timing respects all three at 20 kHz.
- Write the 20 kHz control ISR in compilable C — acquire, compute, output, return, deterministically, with no blocking calls and no allocation. Give the WCET budget per operation in cycles and microseconds, and name the counter or instrument you would measure it with on silicon.
- List every variable shared between interrupt and task context, and the access pattern that makes each safe on this architecture. If floating point appears in interrupt context, state the FPU context-save cost, the stack growth, and the configuration required to make it correct under whichever scheduler you chose.

(c) Control Implementation:

- Design the compensator for 500 Hz closed-loop bandwidth at $\text{PM} \geq 45^\circ$ and $\text{GM} \geq 10 \text{ dB}$. State structure and gains. Give the open-loop gain amplification at the 1.2 kHz resonance implied by the stated ζ , then design the notch — depth, width, and the phase penalty it imposes below the notch frequency.
- Discretise the compensator at 20 kHz. Name the method; if bilinear, state whether you prewarped and at what frequency, and what happens to the notch centre if you do not. Compute the phase lag at the 500 Hz crossover from the zero-order hold and from one sample of computation delay, and subtract both from your margin budget.
- Implement the compensator in fixed point. Give the Q-format of every coefficient and state variable, the accumulator width, the saturation points, and the worst-case input that drives the largest intermediate product — then show it cannot overflow. Specify anti-windup and the bumpless transfer used when the loop closes on acquisition.
- Calculate the peak drive current required for the full-stroke 800 Hz calibration dither. Compare it against the amplifier rating and state the maximum full-stroke frequency this hardware actually supports.

(d) Board Review:

The FSM control board returned from fabrication with the following documented design decisions. Identify every defect, state the field failure mode, and give the fix. Not all of these are defects.

- The ADC DMA receive buffer is placed in DTCM at 0x20000000 "for lowest-latency CPU access", declared as `uint16_t adc_rx[4]`; with D-cache enabled.
- FreeRTOS is configured with NVIC priority grouping at 2 preemption bits and 2 subpriority bits, and the 20 kHz control ISR calls `xQueueSendFromISR` to publish samples to a logging task.
- The independent watchdog is refreshed at the top of the control ISR, "so it can never be starved".
- The ADC reference pin is decoupled with a 2.2 μF X7R and fed from the digital 3V3 rail through a 100 Ω series resistor.
- A ferrite bead isolates the 3V3A analog rail; the ADC bulk decoupling capacitor sits on the digital side of the bead.
- NRST has no external pull-up or capacitor, and BOOT0 is left floating.
- The 25 MHz HSE crystal is fitted with 22 pF load capacitors; the crystal datasheet specifies $C_L = 8$ pF.
- The HV amplifier enable line is driven from a GPIO configured as an input during reset and for the first 40 ms of firmware boot.

Then give two things to the layout engineer before the next revision:

- Which signals must land on timer-capture or DMA-capable pins and the consequence if they do not, plus your grounding and return-path strategy for a board carrying a switching converter, a 100 V amplifier interface, and microvolt strain signals on the same substrate.
- The power-sequencing behaviour your firmware depends on, and exactly what it must verify before it asserts HV amplifier enable.

Question 2: Gimbal Node & Linux Supervisor — Protocols, Interfaces & Partitioning

The mount acquires the satellite from ephemeris and must hold it inside the FSM's 2 mrad capture range for the entire pass — the FSM cannot recover what the mount loses. The supervisor schedules passes, computes pointing from TLEs, carries telemetry to the operations centre, and is the only node with a filesystem and a network stack.

Hardware as specified. Renishaw RESOLUTE absolute encoder per axis over BiSS-C, specified at 23-bit single-turn. Servo drives on EtherCAT running the CiA 402 profile, with a legacy RS-422 serial interface also exposed on the drive. Pointing requirement 1 arcsecond RMS at peak axis rate 3.0 $^\circ/\text{s}$ and peak axis acceleration 1.5 $^\circ/\text{s}^2$, excluding the near-zenith region. Motion controller is an STM32H7-class MCU running a 1 kHz position loop, which must also service an IMU over SPI and a temperature sensor over I²C and publish state every cycle. Supervisor is an NXP i.MX8M Mini running Linux. Inter-node backbone is CAN-FD at 500 kbps arbitration and 8 Mbps data phase, 15 m, 12 nodes, TI TCAN1042 transceivers, 80 MHz CAN kernel clock. Housekeeping sensors around the enclosure sit on an RS-485 multidrop. The terminal reaches the facility equipment room 30 m away over Ethernet, 10 GbE for payload data and 1 GbE for control and telemetry.

(a) Encoder Interface:

- Convert the specified encoder resolution to arcseconds and compare it against the 1 arcsecond RMS requirement. Then verify the specified resolution against the RESOLUTE product range. State whether the part as specified exists, and if not, which resolution you procure and what that does to your error budget.
- Write the BiSS-C master in C: MA clock generation, frame timing, ACK and start bit detection, position field extraction, error and warning bit handling, and CRC verification. State the polynomial, seed, bit order, and whether the result is inverted.

- State whether you implement this bit-banged, on a SPI peripheral, or on a timer plus DMA pattern, and justify against jitter and CPU load at 1 kHz. Give the encoder frame time at your chosen MA clock and the total latency from position sample to actuator command.
- Specify the CRC failure policy: retry count, position-freeze behaviour, what the control loop consumes during a freeze, and the escalation threshold at which you declare the axis failed and hand over to the terminal supervisor.

(b) Buses, External Interfaces & Cycle Budget:

- Produce the cycle budget for the motion node: frame time for the encoder, the IMU, and the temperature sensor at your chosen clock rates, plus control computation and backbone publish. Show the arithmetic and identify what actually dominates the 1 ms period.
- Compute the CAN-FD bit timing for both phases from the 80 MHz kernel clock — prescaler, TSEG1, TSEG2, SJW, and sample point for each. State the TCAN1042 loop delay from its datasheet, compare it against the 8 Mbps data-phase bit time, and state whether transmitter delay compensation is required and at what offset. Then assess whether 8 Mbps over 15 m with 12 nodes is realistic, what physically limits it, and what you would change.
- Decide what runs on RS-422, what runs on RS-485, what runs on CAN-FD, and what runs on Ethernet across this terminal, and defend each assignment. Address termination, fail-safe biasing, isolation, and the ground potential difference you should expect across the 30 m run to the equipment room. State which of these choices you would keep if the drive's EtherCAT interface were unavailable and only the legacy RS-422 remained.
- The temperature sensor sits on I²C and gates a thermal interlock. State the failure mode if that device holds the bus low, whether your firmware can recover it without a power cycle, and what you would do differently given what it gates.

(c) Embedded Linux & Partitioning:

- Justify the split explicitly: state why the 1 kHz motion loop does or does not run on the i.MX8M under Linux. Give the number that supports your argument and name the tool that produces it.
- Configure PREEMPT_RT on the supervisor. State the kernel version and how PREEMPT_RT is obtained for it. Give your boot parameters for core isolation, tick suppression, RCU offload, and IRQ steering, and say what each one buys you.
- For a userspace telemetry process that must not be preempted unpredictably: state the scheduling policy, the priority, the memory-locking call, and the stack pre-faulting step — and state what specifically goes wrong if you omit the memory locking.
- Choose Yocto or Buildroot for this image and justify against reproducibility, field update, and team size. Write the device-tree fragment for one SPI sensor on the i.MX8M including compatible string, chip select and max frequency, and state whether you expose it through spidev or an IIO driver.

(d) Motion Control & Concurrency:

- Partition position, velocity, and current loops between your controller and the EtherCAT drive. Justify the split against the 1 arcsecond budget and the drive's own loop rate.
- Derive velocity and acceleration feedforward from the ephemeris trajectory, and state the tracking error you would have without it at the stated peak rate. Provide compilable C for the position loop and for the interpolator that produces position, velocity and acceleration at 1 kHz from sparse ephemeris points.

- Enumerate every priority inversion and deadlock reachable in this system, including across the CAN-FD backbone and between the motion node and the supervisor. Give the prevention mechanism for each and the code pattern you would ship for the shared bus.
- Provide a runnable simulation of a full LEO pass on both axes. Report RMS and peak tracking error in arcseconds, identify where in the pass the requirement is hardest to meet, and explain the mechanism that makes it hard there.

Grand Challenge: Terminal Integration & Retrospect

Your FSM node from Question 1 and your gimbal and supervisor from Question 2 now integrate with the laser driver node, the QPD receiver front-end, the power distribution board, and the main terminal supervisor. The terminal must acquire a satellite from a cold start, hand pointing from mount to FSM, close the fibre coupling loop, run 10 Gbps traffic, survive atmospheric fades, and do it unattended for 12 months.

Parts A to D ask you to design it. Part E assumes it has been built and integrated, and asks what that process would have taught you.

Part A: Architecture

- Block diagram: every processor, every bus with its cycle time, every power rail from the 48 V input, and every interlock line.
- Defend your node count. State what you merged, what you kept separate, and the deadline or fault-isolation argument behind each decision. Assign bare-metal, RTOS, or Linux per node with a one-line justification, and say where a second scheduler in the system is a liability rather than an asset.
- Specify time synchronisation across nodes: mechanism, achievable accuracy, dominant error source, and how a node detects it has lost sync and what it does next.

Part B: PAT State Machine & Laser Safety

- Deliver the master Pointing, Acquisition and Tracking state machine in compilable C. Every state needs entry and exit actions, quantitative transition criteria, a timeout, and a fallback. Include the coarse-to-fine handoff from mount to FSM, the fibre coupling lock, and the reacquisition path after a 50 ms atmospheric fade.
- Design the laser emission interlock so that no single firmware defect can enable emission. Give the logic, the physical signals, and the truth table. Firmware may permit emission; it must not be able to force it.
- State which interlock conditions are enforced in hardware and which are observed in software, and why each sits where it does.

Part C: Timing, Supervision & FDIR

- End-to-end worst-case latency from beam disturbance to FSM correction, summed across every element in the chain including the QPD front end and any bus hop.
- Watchdog architecture: per-node, plus cross-node heartbeat with miss thresholds, plus the coordinated safe-park when any node goes silent. State what safe-park means specifically for the laser, the FSM, and the gimbal.
- FDIR table: each firmware-visible failure mode with its detection method, detection latency, and whether the response is degrade-and-continue or safe-park.

Part D: Field Update & the Space-Segment Variant

- Bootloader design: partition scheme, integrity verification, rollback trigger and policy, and behaviour when power is lost mid-write. State how you update multiple heterogeneous processors without stranding the terminal in a mixed-version state, and how a node that fails its update is recovered without a site visit.
- The same electronics core must now serve a space-side terminal on a CubeSat bus. The host interface is RS-422 or CAN rather than Ethernet, there is no Linux-class compute budget, power and mass are a fraction of the ground allocation, and there is no service visit ever. State what changes: host interface and protocol, scheduler per node, memory protection and fault recovery, watchdog strategy, and update path.
- State what you would keep identical between the ground and space variants, and why. Then state the one architectural decision in your ground design that you would make differently today knowing the space variant is coming.

Part E: Integration Retrospect

Assume the terminal has been built to your design and integrated. Each of the following was observed during integration. For each: state the root cause, state what should have been designed differently to prevent it, and state what change you make now given the hardware already exists.

- FSM strain feedback shows 20 nm peak-to-peak oscillation at exactly the laser pump converter switching frequency. It was absent when the FSM board was tested standalone. Rank the candidate coupling mechanisms and give, for each, the single measurement that discriminates it from the others.
- The gimbal tracks within budget at low and mid elevation, but following error diverges near the top of every pass, on both axes, for every satellite.
- The FSM node runs correctly for 6 to 40 hours, then stops updating its output while its heartbeat continues. It recovers on power cycle.
- Then give the bring-up sequence you would have used, with the pass/fail gate at each step that would have caught each of the three findings above before full integration. Finish with six quantitative acceptance criteria that gate deployment, each with metric, measurement method, equipment, and threshold.

Deliverables

Submit exactly these. Nothing else is required.

- One PDF or PPTX with all written answers, diagrams, tables and derivations. Hand-drawn scans are acceptable for diagrams.
- Source files, compiling with arm-none-eabi-gcc, build command stated: FSM control ISR and ADC driver, BiSS-C master, gimbal position loop and interpolator, PAT state machine.
- Simulation files, running without errors and regenerating the plots you present: FSM Bode with margins annotated plus closed-loop step response, and two-axis LEO pass tracking error.
- One device-tree fragment, and four tables: Q1 memory map with NVIC priorities, Q1 ISR WCET budget, Q2 cycle budget with CAN-FD bit timing, Grand Challenge FDIR.
- Board review output — defect, failure mode, fix, one line each — and a single reference list of every datasheet and standard used, by document number and revision.

Submission Instructions

What we are actually assessing:

You are not expected to answer every part completely, and some of these questions have no single correct answer. What matters is that you attempt every part, reason clearly about what you do not know, and defend the choices you make. A partial answer with sound reasoning and stated assumptions is worth considerably more to us than a confident answer that ignores a constraint. Where you run out of time or information, say so and state how you would resolve it — that is a legitimate engineering answer and we will read it as one. Where you disagree with a specification or with the way a question is framed, say that too. Attempt everything, defend what you can, and let the working show. Effort and judgment are what we are reading for.

Timeline:

- Recommended: Questions 1 and 2 first (12–16 hours), then the Grand Challenge (8–10 hours).
- Total expected effort: 20–26 hours. You will be given 7 days.

Format:

- Submit as a single archive. Code and simulations as separate files, not pasted images.
- For each numeric answer, mark it calculated, simulated, measured, or estimated.
- Reference all datasheets, reference manuals, application notes and standards by document number and revision.
- If using AI to assist, state which model (e.g. ChatGPT Pro 5.1, Claude Opus 4.5, Sonnet 4.5, Gemini 3, etc.) and on which sub-parts. Use is permitted; undisclosed use is disqualifying.
- Shortlisted candidates will discuss and defend this submission in a live technical session.